

Decomposições: SVD, Autovalores e PCA

As direções que uma matriz não muda de sentido — e por que comprimir dados é encontrá-las

Autovalores, autovetores e a Decomposição em Valores Singulares (SVD) não são um capítulo isolado de álgebra linear — são a mesma ideia aplicada a matrizes quadradas e retangulares: encontrar as direções especiais em que uma transformação linear age como um simples esticamento. Este material constrói essa ideia do zero e mostra que PCA, compressão de imagem, sistemas de recomendação e embeddings são a mesma conta.

Nível: Intermediário — requer o módulo anterior (vetores, matrizes, projeção)

Pre-requisitos: Vetores, Matrizes e Projeções (módulo 1 deste tema)

Carga sugerida: 8 a 10 horas (leitura + 3 notebooks)

Notebooks: 01-autovalores-e-autovetores · 02-svd-e-posto-baixo · 03-pca-do-zero · 99-exercicios

Autoria: Material do curso de ML & DL

Versao: 1.0

Sumário

1. Por que este módulo existe	3
1.1 O que você vai conseguir fazer ao final	3
2. Autovalores e autovetores: direções que não mudam de sentido	4
2.1 O teorema espectral: o resultado mais usado de toda a álgebra linear aplicada	4
2.2 Interpretação geométrica: elipses e esticamento	5
2.3 Power iteration: como o computador de fato encontra o maior autovalor	5
3. SVD: a mesma ideia para matrizes retangulares	6
3.1 A ponte com autovalores	6
3.2 Posto, de novo	7
4. Aproximação de posto baixo: comprimir mantendo o essencial	8
5. PCA: SVD com um passo de pré-processamento	9
5.1 Via covariância (a definição clássica)	9
5.2 Via SVD dos dados centralizados (o que a prática usa)	9
5.3 Variância explicada e a escolha de k	9
5.4 Whitening: um subproduto útil	10
6. Erros que custam caro — checklist	11
7. Para ir além	12

1. Por que este módulo existe

O módulo anterior mostrou que uma matriz é uma máquina que transforma vetores. Esta pergunta ficou em aberto: **transforma como, exatamente?** Uma matriz pode esticar em uma direção, comprimir em outra, girar, refletir — tudo ao mesmo tempo. Autovalores e autovetores respondem a essa pergunta encontrando as direções em que a transformação é mais simples possível: um esticamento puro, sem rotação. A SVD generaliza a mesma pergunta para matrizes que nem são quadradas — o que cobre praticamente todo dado tabular de machine learning.

ANALOGIA

Imagine amassar uma bola de massa de modelar em uma prensa. A prensa estica a massa em algumas direções e comprime em outras. Os **autovetores** (ou, no caso geral, os **vetores singulares**) são os eixos da bola que continuam apontando para o mesmo lugar depois da prensa — só mudam de comprimento. Os **autovalores** (ou **valores singulares**) dizem o quanto cada eixo esticou ou encolheu.

1.1 O que você vai conseguir fazer ao final

- Explicar o que um autovalor e um autovetor realmente significam, geometricamente.
- Diagonalizar uma matriz simétrica e explicar por que essa forma é especial.
- Construir a SVD de qualquer matriz e ler o que cada peça (U , Σ , V) faz.
- Justificar, com o teorema de Eckart-Young, por que a SVD é a melhor compressão possível de uma matriz em posto baixo.
- Derivar o PCA a partir da covariância (autovalores) e a partir da SVD dos dados — e saber por que a segunda via é a que se usa na prática.

2. Autovalores e autovetores: direções que não mudam de sentido

Definição. Dado $A \in \mathbb{R}^{n \times n}$, um vetor não-nulo v é um **autovetor** de A com **autovalor** λ se:

$$Av = \lambda v$$

Ou seja: aplicar A a v produz o mesmo vetor, só reescalado por λ . A direção não muda — só o comprimento (e o sentido, se $\lambda < 0$).

DEFINICAO

Autovalores são as raízes do **polinômio característico** $\det(A - \lambda I) = 0$. Para cada autovalor, os autovetores associados formam o **autoespaço**: o conjunto de todas as direções que A trata daquele jeito.

Na prática, ninguém calcula autovalores resolvendo o polinômio característico à mão além de matrizes 2×2 — o custo numérico explode e o método é instável. `np.linalg.eig` (matrizes gerais) e `np.linalg.eigh` (matrizes simétricas) usam algoritmos iterativos que fazem isso de forma estável.

ARMADILHA COMUM

Use sempre `eigh` quando a matriz for simétrica (como toda matriz de covariância é). `eig` genérico ignora essa estrutura, é mais lento, pode devolver autovalores complexos por erro de arredondamento e não garante autovetores ortogonais. `eigh` garante autovalores reais e ordenáveis, e autovetores ortonormais — porque explora o **teorema espectral**.

2.1 O teorema espectral: o resultado mais usado de toda a álgebra linear aplicada

Toda matriz simétrica real A (isto é, $A = A^T$ — e toda matriz de covariância, correlação e kernel é simétrica) pode ser escrita como:

$$A = Q\Lambda Q^T$$

onde Q é uma matriz **ortogonal** (colunas são autovetores unitários e mutuamente ortogonais, $Q^T Q = I$) e Λ é diagonal com os autovalores. Em código, uma matriz diagonal como Λ nunca é escrita como ambiente matemático em bloco — é mais claro escrevê-la como código:

```
# Lambda = diag(lambda_1, ..., lambda_n), fora da diagonal tudo zero
Lambda = np.diag([lam1, lam2, lam3])
```

FORMULARIO

Consequências do teorema espectral que valem memorizar:

- Toda matriz simétrica tem autovalores **reais** (nunca complexos). - Os autovetores podem sempre ser escolhidos **ortogonais entre si**. - A é **positiva semidefinida** (todo $x^T A x \geq 0$) se e somente se todos os autovalores são ≥ 0 . Matrizes de covariância são sempre positivas semidefinidas — variância nunca é negativa. - $\det(A) = \prod_i \lambda_i$ e $\text{tr}(A) = \sum_i \lambda_i$: o traço e o determinante são resumos baratos do espectro inteiro.

NO MERCADO

Toda vez que alguém fala em "decompor a variância em componentes independentes", "encontrar as direções de maior variação" ou "regularizar autovalores pequenos" (como faz a Ridge), está falando do teorema espectral aplicado à matriz de covariância. É o resultado que sustenta PCA, análise fatorial e a interpretação geométrica da regularização.

2.2 Interpretação geométrica: elipses e esticamento

Para uma matriz simétrica positiva definida, o conjunto $\{x : x^T A x = 1\}$ é uma **elipse** (ou elipsoide, em mais dimensões). Os eixos da elipse apontam exatamente na direção dos autovetores, e o comprimento de cada eixo é $1/\sqrt{\lambda_i}$. Autovalores grandes correspondem a eixos **curtos** — a função cresce rápido naquela direção — e autovalores pequenos correspondem a eixos **longos** — a função quase não muda. Essa é a imagem geométrica que explica todo o problema de condicionamento discutido no próximo módulo: quando a razão entre o maior e o menor autovalor é grande, a elipse fica esticadíssima, e otimizar sobre ela fica difícil.

2.3 Power iteration: como o computador de fato encontra o maior autovalor

Um algoritmo simples e revelador — muitos algoritmos de recomendação e o próprio PageRank do Google são variações dele:

- 1 Comece com um vetor aleatório v_0 .
- 2 Repita $v_{k+1} = A v_k / \|A v_k\|$.
- 3 v_k converge para o autovetor de **maior autovalor absoluto**, e $\|A v_k\|$ converge para esse autovalor.

NOTA

Por que converge: escreva v_0 na base dos autovetores. A cada multiplicação por A , a componente ao longo do maior autovalor cresce mais rápido (ou encolhe mais devagar) que as demais, proporcionalmente a $(\lambda_1/\lambda_2)^k$. Depois de normalizar a cada passo, todas as outras componentes desaparecem exponencialmente rápido — a velocidade de convergência depende de quão separados estão λ_1 e λ_2 .

3. SVD: a mesma ideia para matrizes retangulares

Autovalores só existem para matrizes quadradas — e mesmo assim, só a versão simétrica garante tudo de bom que vimos. A maioria dos dados de machine learning vem em uma matriz X de n observações por p features, que **não é quadrada**. A **Decomposição em Valores Singulares (SVD)** resolve isso.

Toda matriz $X \in \mathbb{R}^{n \times p}$ pode ser escrita como:

$$X = U \Sigma V^T$$

- $U \in \mathbb{R}^{n \times n}$: ortogonal — suas colunas são os **vetores singulares à esquerda**.

- $V \in \mathbb{R}^{p \times p}$: ortogonal — suas colunas são os **vetores singulares à direita**.

- $\Sigma \in \mathbb{R}^{n \times p}$: retangular, zero fora da diagonal

principal, com os **valores singulares** $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$ na diagonal, em ordem decrescente.

ANALOGIA

A SVD diz que **toda** transformação linear, por mais estranha que pareça, é a composição de três passos simples: uma rotação/reflexão (V^T), um esticamento puro ao longo dos eixos coordenados (Σ), e outra rotação/reflexão (U). Não existe transformação linear que fuja dessa receita — é por isso que a SVD é chamada de o resultado mais importante da álgebra linear numérica.

3.1 A ponte com autovalores

A SVD de X e a decomposição espectral de $X^T X$ e XX^T são a mesma informação vista de dois ângulos:

$$X^T X = V \Sigma^T \Sigma V^T, \quad XX^T = U \Sigma \Sigma^T U^T$$

- As colunas de V são os **autovetores de $X^T X$** .
- As colunas de U são os **autovetores de XX^T** .
- Os valores singulares são as **raízes quadradas dos autovalores** (não-nulos)

de $X^T X$ (ou, equivalentemente, de XX^T): $\sigma_i = \sqrt{\lambda_i}$.

ARMADILHA COMUM

Nunca calcule a SVD dessa forma na prática — formar $X^T X$ eleva o número de condição ao quadrado, exatamente como no módulo anterior. `np.linalg.svd` trabalha em X diretamente, com algoritmos numericamente estáveis. A relação acima serve para **entender**, não para **calcular**.

3.2 Posto, de novo

O número de valores singulares **não-nulos** é o posto de X . Como os σ_i vêm ordenados, a SVD também dá, de graça, um **ranking de importância** das direções: as primeiras capturam mais estrutura, as últimas capturam menos (ou apenas ruído).

4. Aproximação de posto baixo: comprimir mantendo o essencial

Se você usar só os k primeiros valores singulares e os vetores correspondentes:

$$X_k = \sum_{i=1}^k \sigma_i u_i v_i^\top$$

obtem uma matriz X_k de **posto** k — uma versão comprimida de X .

FORMULARIO

Teorema de Eckart-Young. Entre todas as matrizes de posto no máximo k , X_k (construída com os k maiores valores singulares) é a que minimiza $\|X - X_k\|$, tanto na norma de Frobenius quanto na norma espectral. Não existe compressão de posto k melhor que essa — a SVD é **ótima**, não apenas "boa".

O erro dessa compressão tem fórmula fechada:

$$\|X - X_k\|_F^2 = \sum_{i=k+1}^r \sigma_i^2$$

— a soma dos quadrados dos valores singulares descartados. Isso permite decidir k olhando para a **energia explicada**: $\left(\sum_{i \leq k} \sigma_i^2\right) / \left(\sum_i \sigma_i^2\right)$, o análogo exato da variância explicada em PCA.

NO MERCADO

Compressão de imagem, remoção de ruído (os valores singulares pequenos costumam ser ruído, não sinal), sistemas de recomendação por fatoração de matrizes, e a etapa de redução de dimensionalidade em pipelines de NLP clássico (LSA — *Latent Semantic Analysis*) são todas aplicações diretas de Eckart-Young. "Encontrar a estrutura latente" quase sempre significa "truncar a SVD".

5. PCA: SVD com um passo de pré-processamento

Análise de Componentes Principais responde: qual é a direção de maior variância nos dados, a segunda maior (ortogonal à primeira), e assim por diante?

5.1 Via covariância (a definição clássica)

- 1 Centralize os dados: $\tilde{X} = X - \bar{X}$ (subtraia a média de cada coluna).
- 2 Calcule a matriz de covariância: $C = \frac{1}{n-1} \tilde{X}^\top \tilde{X}$.
- 3 Diagonalize $C = Q\Lambda Q^\top$ (teorema espectral — C é simétrica).
- 4 As colunas de Q , ordenadas pelo autovalor decrescente, são os **componentes principais**. O autovalor λ_i é a variância dos dados projetados naquele componente.

5.2 Via SVD dos dados centralizados (o que a prática usa)

Aplicando a SVD diretamente em $\tilde{X} = U\Sigma V^\top$:

- V é idêntica aos componentes principais Q de cima.
- $\lambda_i = \sigma_i^2 / (n - 1)$ — a variância explicada vem direto dos valores singulares ao quadrado.
- As **coordenadas dos dados nos componentes** (os *scores*) são $U\Sigma$, sem nunca precisar formar C .

ARMADILHA COMUM

Formar a matriz de covariância explicitamente e diagonalizá-la é exatamente o erro de "formar $X^\top X$ " do módulo anterior: perde precisão numérica e, se p (número de features) for muito maior que n , C vira gigantesca e computacionalmente cara desnecessariamente. `sklearn.PCA` usa SVD por padrão pelas mesmas razões que `LinearRegression` usa QR/SVD.

5.3 Variância explicada e a escolha de k

$$\text{variância explicada por } k \text{ componentes} = \frac{\sum_{i=1}^k \lambda_i}{\sum_{i=1}^p \lambda_i}$$

É comum escolher k pelo "cotovelo" do gráfico de variância explicada acumulada, ou por um limiar arbitrário (90%, 95%). Nenhum dos dois é uma lei da física — são heurísticas, e a escolha depende do que você fará com os componentes depois.

ARMADILHA COMUM

PCA **maximiza variância**, não relevância preditiva. Uma feature com pouca variância pode ser exatamente a que separa as classes que você quer prever, e o PCA vai descartá-la se ela não contribuir muito para a variância total. PCA é uma ferramenta de **compressão não supervisionada**, não de seleção de features para uma tarefa específica.

ARMADILHA COMUM

PCA é sensível à **escala** das variáveis. Uma feature em reais (variando na casa dos milhares) domina completamente uma feature em proporção (variando entre 0 e 1) só porque tem variância numericamente maior — não porque é mais importante. Sempre padronize (`StandardScaler`) antes de aplicar PCA em features de escalas diferentes.

5.4 Whitening: um subproduto útil

Dividir cada componente pelo seu desvio-padrão (σ_i) depois de projetar produz dados com covariância igual à identidade — sem correlação entre variáveis e variância unitária em todas as direções. É o pré-processamento clássico antes de certos algoritmos que assumem features não-correlacionadas, e a mesma ideia reaparece em normalizações usadas dentro de redes neurais.

6. Erros que custam caro — checklist

- Usar `eig` genérico em vez de `eigh` para matrizes simétricas.
- Formar $X^T X$ ou a matriz de covariância explicitamente antes de decompor, em vez de aplicar SVD direto em X .
- Aplicar PCA sem padronizar features de escalas muito diferentes.
- Interpretar variância explicada como relevância preditiva.
- Esquecer que os sinais dos autovetores/vetores singulares são arbitrários — se seu resultado tem sinal trocado em relação a uma referência, não é um bug.
- Escolher k (posto da aproximação, número de componentes) sem olhar para o gráfico de valores singulares ou variância explicada.
- Interpretar os componentes principais como se tivessem significado causal ou de negócio automático — eles são combinações lineares que maximizam variância, e precisam de trabalho adicional para virar uma narrativa.

7. Para ir além

- Strang, *Introduction to Linear Algebra* — os capítulos sobre autovalores e SVD, com a mesma abordagem geométrica deste módulo.
- Trefethen & Bau, *Numerical Linear Algebra*, capítulos 4 a 6 — por que a SVD é o algoritmo mais importante da área.
- Jolliffe, *Principal Component Analysis* — o tratamento de referência, incluindo os casos em que PCA falha.
- 3Blue1Brown, *Essence of Linear Algebra*, vídeo sobre autovalores e autovetores — a intuição visual antes da álgebra.